

CC 2.1.3 : Les « commands » custom vivent désormais dans `.claude/skills/` avec `disable-model-invocation: true`. La syntaxe `/nom` et le comportement sont identiques.

Principe
Une slash command est un fichier Markdown qui définit un workflow. Claude l'exécute comme s'il avait reçu ce texte en prompt, avec la possibilité de passer des arguments dynamiques.

Emplacement des fichiers

```
.claude/skills/      # Commandes projet (équipe)
├─ commit.md        → /commit
├─ review-pr.md     → /review-pr    └─ tech/
├─ deploy.md        → /tech:deploy
└─ ...
~/.claude/skills/   # Commandes globales (perso)
├─ release.md       → /release
├─ sync.md          → /sync
```

Les commandes globales sont disponibles dans toutes les sessions, quel que soit le projet.

Invocation et arguments

```
/commit           # Aucun argument
/tech:deploy production # Argument positionnel
/release minor     # Bump de version
```

Dans le fichier Markdown, les arguments sont accessibles via des variables :

```
Déployer en environnement : $ARGUMENTS[0]
Avec la stratégie : $ARGUMENTS[1]
Si aucun argument fourni : utiliser "staging" par défaut.
```

La syntaxe `$0` , `$1` est équivalente à `$ARGUMENTS[0]` , `$ARGUMENTS[1]` . L'ancienne notation avec point (`$ARGUMENTS.0`) est dépréciée depuis v2.1.19.

Ajouter `argument-hint` dans le frontmatter pour afficher un placeholder dans le menu : `argument-hint: "<env> [--dry-run]"` → le picker affiche `/deploy <env> [--dry-run]` .

Structure recommandée

```
# Nom de la Commande  ## Objectif
Ce que fait cette commande en une phrase.  ## Processus
1. **Étape 1** – instructions détaillées  2.
**Étape 2** – instructions détaillées  3.
**Étape 3** – instructions détaillées  ## Arguments
- $0 : environnement cible (défaut: staging)
## Format de sortie  Description du résultat attendu.
```

Commandes natives récentes

Commande	Usage
<code>/goal [condition]</code>	Mode autonome multi-tours : Claude travaille jusqu'à ce que la condition soit remplie,

Commande	Usage
	avec overlay temps/tours/tokens (v2.1.139)
<code>/scroll-speed</code>	Curseur interactif pour régler la vitesse de défilement de la souris avec aperçu en direct (v2.1.139)

Commandes utiles à créer

Commande	Usage
<code>/commit</code>	Commit conventionnel automatique
<code>/review-pr</code>	Revue de PR selon critères projet
<code>/release [patch minor major]</code>	Bump version + changelog
<code>/sync</code>	Vérification cohérence multi-fichiers
<code>/security-check</code>	Scan rapide de la config

Command vs Skill vs Agent

Mécanisme	Quand l'utiliser
Command	Workflow ponctuel, procédure à suivre
Skill	Connaissance réutilisable + ressources embarquées
Agent	Spécialiste récurrent avec mémoire propre

Une command ne peut pas embarquer des fichiers de référence supplémentaires — pour ça, utiliser un skill. Une command n'a pas de mémoire persistante — pour ça, utiliser un agent.

Conseils pratiques

Une commande, une responsabilité. Dès qu'une commande fait deux choses distinctes, la découper. Nommer avec un verbe d'action en premier (`commit` , `release` , `sync`) pour que l'intention soit immédiatement lisible dans la liste des commandes disponibles.